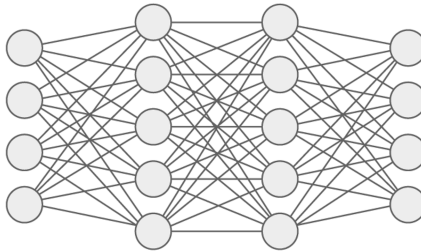


Deep Learning and Artificial Neural Networks

Recurrent Neural Networks



Contents

- 1 Sequential Data
- 2 Notation in Sequence Models
- 3 Recurrent Neural Network
- 4 Language Model and Sequence Generation

Sequential Data

Recurrent Neural Network (RNN) is a type of neural network designed for processing **sequential data**.

Sequence data refers to a type of data where the order of elements is significant, with the elements typically arranged in a specific sequence or chronological order. Each element in the sequence is related to the ones that precede and follow it, and their order provides context, meaning, and structure to the data.

Key characteristics of sequence data:

- **Order matters:** In sequence data, the arrangement of elements is crucial. Changing the order of elements can lead to different meanings or interpretations. For example, in natural language processing, the order of words in a sentence determines its meaning.
- **Temporal or sequential relationship:** Sequence data often exhibits a temporal or sequential relationship, meaning that the elements are typically collected or observed in a specific order over time. For instance, time series data, such as stock prices, records values at different time points.
- **Variable length:** Sequences can have variable lengths. In natural language, sentences may vary in length, and in genomics, DNA sequences can differ in length depending on the organism.

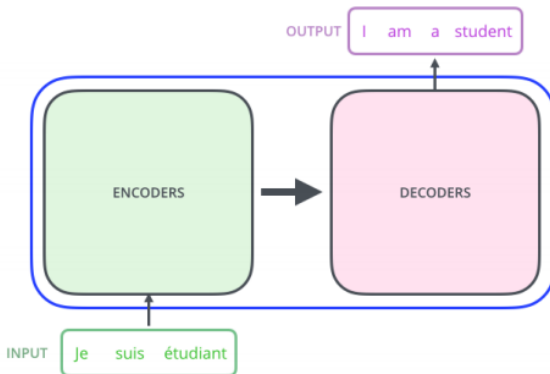
Examples of Sequence Data

Time series data: is a common type of sequential data where measurements are taken at regular intervals over time. Examples include stock prices, temperature readings, and sensor data.



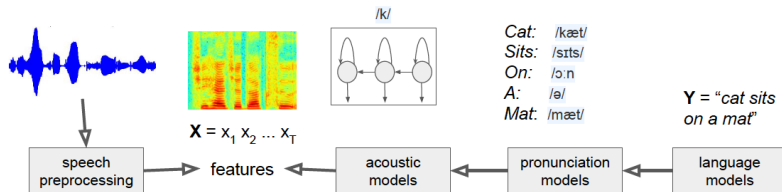
Examples of Sequence Data

Natural language text: textual data, such as sentences or paragraphs, are inherently sequential, so the order cannot be ignored or treated as irrelevant. The order of words is crucial for representing meaning, establishing context, and defining relationships between words.



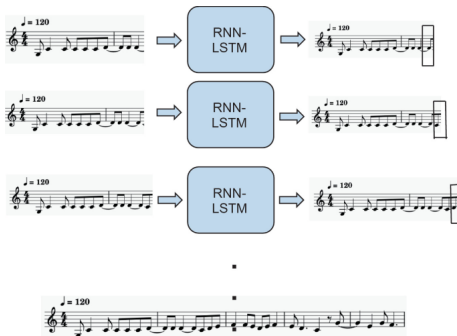
Examples of Sequence Data

Speech signals: audio signals from spoken language are sequential. Phonemes, syllables, and words are organized sequentially to form sentences.



Examples of Sequence Data

Music: Musical notes in a composition are arranged sequentially to create melodies, harmonies, and rhythms.



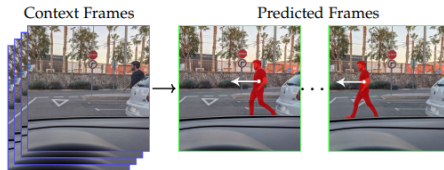
Examples of Sequence Data

DNA Sequences: In genetics, DNA sequences represent the arrangement of nucleotides in a specific order, carrying genetic information.



Examples of Sequence Data

Video frames: in video data, each frame is a snapshot of a scene at a specific time. The sequence of frames creates the illusion of motion.



Notation in Sequence Models

- **Input sequence:** $x^{(1)}, x^{(2)}, \dots, x^{(T_x)}$

Example: For a sentence “I love AI”, the input sequence could be represented as $x^{(1)} = \text{“I”}$, $x^{(2)} = \text{“love”}$, $x^{(3)} = \text{“AI”}$, where $T_x = 3$.

- **Output sequence:** $y^{(1)}, y^{(2)}, \dots, y^{(T_y)}$

Example: For a translation task, if the output is “Volim AI”, the output sequence would be $y^{(1)} = \text{“Volim”}$, $y^{(2)} = \text{“AI”}$, where $T_y = 2$.

- **Sequence length:** T_x (input length), T_y (output length)

Example: In the above examples, $T_x = 3$ for the input sequence and $T_y = 2$ for the output sequence.

- **Training example notation:** $x^{(i)}$ for the i -th training example

Example: If there are multiple sentences in the dataset, $x^{(1)}$ could represent the first sentence (“I love AI”), and $x^{(2)}$ could represent the second sentence (“AI is fun”).

- **Variable sequence lengths:** $T_x^{(i)}$ for different training samples

Example: For two training examples, $T_x^{(1)} = 3$ for the sentence “I love AI” and $T_x^{(2)} = 4$ for the sentence “AI is very fun”.

Example: Named-Entity Recognition (NER)

Named-Entity Recognition (NER) is a Natural Language Processing (NLP) technique used to identify and classify entities in text into predefined categories.

- **Task:** Identify people's names in text.
 - Example sentence: *"Harry Potter and Hermione Granger invented a new spell."*
- **Output:** Binary labels indicating whether each word is part of a person's name.
- **Example:**
 - Input sequence:

$$x = [x^{(1)}, x^{(2)}, x^{(3)}, \dots, x^{(T_x)}]$$

- $x^{(1)} = \text{"Harry"}$
 - $x^{(2)} = \text{"Potter"}$
 - $x^{(3)} = \text{"and"}$
 - ...
 - $T_x = 9$ (length of the input sequence).
- Output sequence:

$$y = [y^{(1)}, y^{(2)}, y^{(3)}, \dots, y^{(T_y)}]$$

- $y^{(1)} = 1$ (indicating "Harry" is part of a name)
- $y^{(2)} = 1$ (indicating "Potter" is part of a name)
- $y^{(3)} = 0$ (indicating "and" is not part of a name)
- ...
- $T_y = 9$ (length of the output sequence, same as T_x).

Representing Words: Vocabulary and One-Hot Vectors

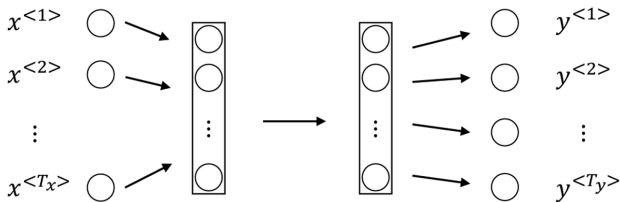
- **Vocabulary (Dictionary):** A list of unique words used in the model.
 - Example: ["a", "Aaron", "and", ..., "Harry", "Potter", ..., "Zulu"]
 - Size: Typically 10,000–50,000 words; larger models use 1M+ words.
- **One-Hot Encoding:** Represent each word as a vector.
 - Example: "Harry" $\rightarrow [0, \dots, 1, \dots, 0]$, where 1 is at position 4075.
 - Dimensionality: Equal to vocabulary size (e.g., 10,000).
- **Example Representation:**
 - $x^{(1)}$: "Harry" $\rightarrow [0, \dots, 1, \dots, 0]$ (1 at position 4075).
 - $x^{(2)}$: "Potter" $\rightarrow [0, \dots, 1, \dots, 0]$ (1 at position 6830).

Handling unknown words

- **Problem:** What if a word is not in the vocabulary?
- **Solution:** Introduce a special token: **UNK** (Unknown Word).
- **Example:**
 - Input: "Quidditch" (not in vocabulary).
 - Representation: $x = [\text{UNK}] \rightarrow [0, \dots, 1, \dots, 0]$ (1 at the UNK position).
- **Importance:** Ensures the model can handle unseen words during inference.

Why Standard Neural Networks Fail for Sequence Data?

- **Initial idea:** Use a standard neural network to map input x to output y .
 - Example: Take nine one-hot vectors as inputs (e.g., words in a sentence).
 - Pass them through hidden layers and output binary labels (e.g., whether each word is part of a person's name).



Why Standard Neural Networks Fail for Sequence Data?

- **Problem 1: Variable input and output lengths**

- Input and output sequences can have different lengths across examples.
- Example: One sentence may have 9 words, another 15, and another just 3.
- Padding or zero-padding could address this, but it's not an ideal solution.

- **Problem 2: Lack of feature sharing across positions**

- A standard neural network doesn't share features learned across different positions in the text.
- Example: If the network learns that "Harry" in position 1 indicates a person's name, it won't automatically generalize this knowledge to "Harry" appearing in position 5 or 10.
- This is similar to how convolutional neural networks (CNNs) share features across different parts of an image.

Why Standard Neural Networks Fail for Sequence Data?

- **Additional Issue: High parameter count**

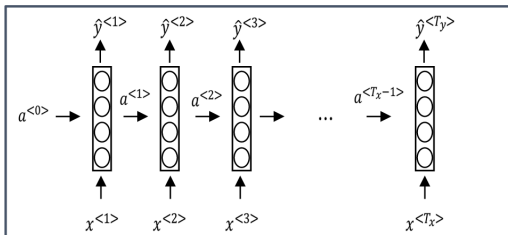
- Each input word is represented as a high-dimensional one-hot vector (e.g., 10,000 dimensions).
- This leads to a very large input layer and weight matrix, resulting in an enormous number of parameters.
- Example: For a sentence with 9 words, the total input size would be $9 \times 10,000 = 90,000$, which is computationally expensive.

- **Solution: Recurrent Neural Networks (RNNs)**

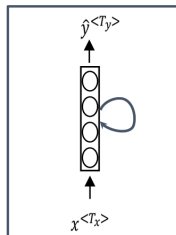
- RNNs address both problems:
 - They handle variable-length sequences naturally.
 - They share features across different positions in the sequence.

Introduction to Recurrent Neural Networks (RNNs)

- **Key idea:** RNNs process sequences one step at a time, maintaining a hidden state that captures information from previous steps.
- **Unrolled representation:**
 - At each time step t , the RNN takes an input $x^{(t)}$ and produces an output $\hat{y}^{(t)}$.
 - Hidden state $a^{(t)}$ is passed to the next time step.
- **Initialization:**
 - Hidden state at $t = 0$: $a^{(0)} = \vec{0}$ (vector of zeros).
 - Parameters are shared across all time steps.



Unrolled representation



Compact representation

Forward Propagation in RNNs

- **Hidden State Update:**

$$a^{\langle t \rangle} = g(W_{aa}a^{\langle t-1 \rangle} + W_{ax}x^{\langle t \rangle} + b_a)$$

where:

- W_{aa} : Weight matrix for hidden-to-hidden connections.
- W_{ax} : Weight matrix for input-to-hidden connections.
- b_a : Bias term.
- g : Activation function (commonly $\tanh$).

- **Output Prediction:**

$$\hat{y}^{\langle t \rangle} = g'(W_{ya}a^{\langle t \rangle} + b_y)$$

where:

- W_{ya} : Weight matrix for hidden-to-output connections.
- b_y : Bias term.
- g' : Output activation function (e.g., sigmoid for binary classification).

Limitations of Unidirectional RNNs

- **Key issue:** Unidirectional RNNs only use information from earlier in the sequence.
- **Example Problem:**
 - Sentence: "He said Teddy Roosevelt was a great president."
 - Ambiguity: Is "Teddy" part of a person's name or referring to teddy bears?
 - Solution: Need information from later in the sequence to resolve ambiguity.
- **Proposed Solution:** Bidirectional RNNs (BRNNs).

Simplified Notation for RNNs

To simplify their mathematical representation, we introduce a compact notation.

- **Combined weight matrices:**

- Instead of maintaining separate weight matrices for the hidden-to-hidden transformation W_{aa} and the input-to-hidden transformation W_{ax} , we concatenate them into a single matrix:

$$W_a = [W_{aa}, W_{ax}]$$

- This allows the recurrence relation to be expressed more concisely:

$$a^{(t)} = g(W_a[a^{(t-1)}; x^{(t)}] + b_a)$$

where:

- $a^{(t)}$ is the hidden state at time step t ,
- $g(\cdot)$ is the activation function (commonly tanh or ReLU),
- $x^{(t)}$ is the input at time step t ,
- b_a is the bias term.

Simplified Notation for RNNs

- **Output equation:**

- The network produces an output $\hat{y}^{(t)}$ at each time step based on the current hidden state:

$$\hat{y}^{(t)} = g'(W_y a^{(t)} + b_y)$$

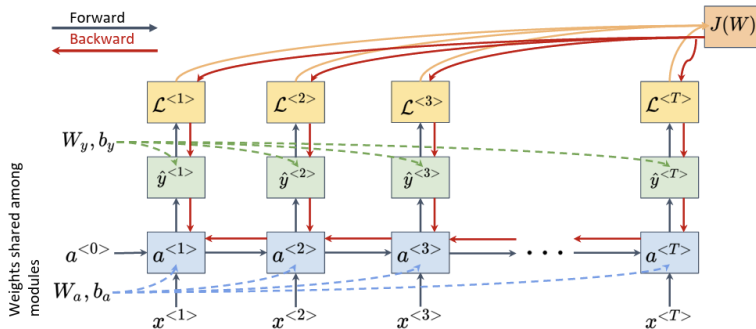
where:

- W_y is the weight matrix for mapping the hidden state to the output,
- $g'(\cdot)$ is the activation function (often softmax for classification or linear for regression),
- b_y is the output bias.

- **Advantages of simplified notation:**

- **Reduces clutter:** Helps keep equations compact and readable, especially when extending to deeper or bidirectional RNNs.
- **Easier generalization:** This notation makes it straightforward to extend RNNs to advanced architectures like:
 - Long Short-Term Memory (LSTM), which introduces memory cells and gating mechanisms.
 - Gated Recurrent Units (GRU), which simplify LSTMs while maintaining performance.
 - Transformer-based models, where RNN concepts help in understanding sequence processing.

Backpropagation Through Time



$$\mathcal{L}^{<t>}(\hat{y}^{<t>}, y^{<t>}) = y^{(i)} \log(f(x^{(i)})) + (1 - y^{(i)}) \log(1 - f(x^{(i)})) \quad \text{Cross-entropy loss for each output}$$

$$J(W) = \sum_{t=1}^T \mathcal{L}^{<t>}(\hat{y}^{<t>}, y^{<t>}) \quad \text{Total cost}$$

Backpropagation Through Time (BPTT) in RNNs

- **Total cost function:** The sum of cross-entropy losses across all time steps:

$$J(W) = \sum_{t=1}^T \mathcal{L}^{(t)}(\hat{y}^{(t)}, y^{(t)})$$

- **Loss function at each step:**

$$\mathcal{L}^{(t)} = y^{(i)} \log f(x^{(i)}) + (1 - y^{(i)}) \log(1 - f(x^{(i)}))$$

Measures the difference between prediction $\hat{y}^{(t)}$ and true output $y^{(t)}$.

- **Weight matrices:**

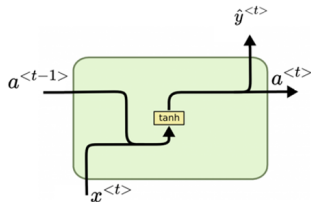
- W_a, b_a update hidden states: $a^{(t)} = g(W_a[a^{(t-1)}; x^{(t)}] + b_a)$.
- W_y, b_y transform hidden states into outputs: $\hat{y}^{(t)} = g'(W_y a^{(t)} + b_y)$.

- **Backpropagation through time (BPTT):**

- Errors (red arrows) are propagated backward through time.
- The same weights W_a, b_a are updated across all time steps.
- Helps train deep sequence models but suffers from vanishing gradients.

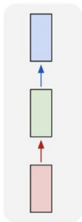
Types of RNN

- One to One: "Vanilla" neural network.
- One to Many: Music generation.
- Many to One: Sentiment classification.
- Many to Many: Translation.

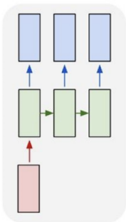


$$a^{<t>} = g(W_a[a^{<t-1>}, x^{<t>}] + b_a)$$

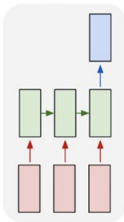
one to one



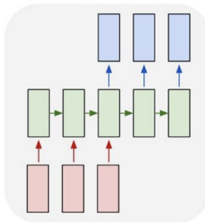
one to many



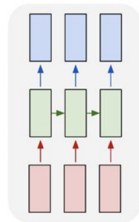
many to one



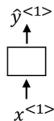
many to many



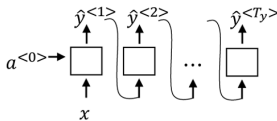
many to many



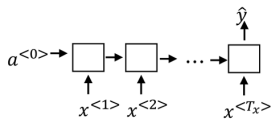
Types of RNN



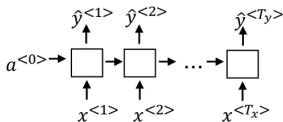
One to one



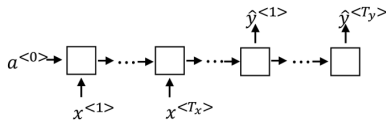
One to many



Many to one



Many to many



Many to many

Introduction to Language Modeling

● What is a Language Model?

- A language model estimates the probability of a given sentence.
- Consider a speech recognition system distinguishing between two similar-sounding sentences:
 - **Sentence 1:** "The apple and pear salad."
 - **Sentence 2:** "The apple and pair salad."
- A good language model assigns a higher probability to the sentence that is more likely in natural language:

$$P(\text{"The apple and pair salad"}) = 3.2 \times 10^{-13}$$

$$P(\text{"The apple and pear salad"}) = 5.7 \times 10^{-10}$$

- Since "pear" (a fruit) makes more sense in this context than "pair," the model assigns it a higher probability, helping the system select the correct sentence.
- **Goal:** Given a sentence $y^{(1)}, y^{(2)}, \dots, y^{(T_y)}$, estimate its probability:

$$P(y^{(1)}, y^{(2)}, \dots, y^{(T_y)})$$

Building a Language Model with RNNs

- **Training data:** A language model requires a **large corpus** (collection of text) from the target language (e.g., English sentences).
- **Tokenization:** Convert words into indices or one-hot vectors based on a predefined vocabulary. Optionally, decide whether to include punctuation as tokens.
- Add an **⟨EOS⟩** (End-of-Sentence) token to mark sentence boundaries, helping the model recognize when a sentence ends.
- Replace unknown words (not in the vocabulary) with a special **⟨UNK⟩** token, allowing the model to generalize beyond its vocabulary size.

Building a Language Model with RNNs

- **Example sentences:**

- **Sentence 1:** "Cats average 15 hours of sleep a day."
- **Tokenized representation:**

[cats, average, 15, hours, of, sleep, a, day, **⟨EOS⟩**]

- **Sentence 2:** "The Egyptian Mau is a breed of cat."
- **Tokenized representation:**

[The, Egyptian, **⟨UNK⟩**, is, a, breed, of, cat, **⟨EOS⟩**]

- **Note:**

- The period is omitted in these examples, but it could be included as a separate token if desired.
- The word "Mau" is not in the vocabulary, so it is replaced with the special token **UNK**.

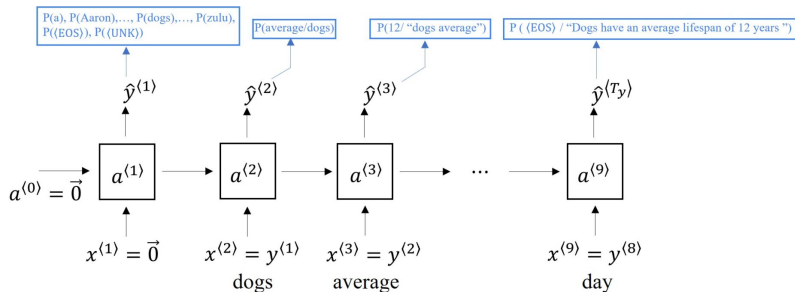
RNN Architecture for Language Modeling

Objective: Train an RNN to predict the next word in a sequence, one word at a time.

Example Sentence: "Dogs have an average lifespan of 12 years."

Process:

- The RNN processes the sentence word by word.
- At each step, the model predicts the next word based on previous words.
- Training helps the model learn how likely different words are in context.



Step-by-Step Processing in an RNN

- **Input: zero vector**

- At the start, the model has no prior information.
- We use an initial input $x^{\langle 1 \rangle} = \vec{0}$ (zero vector) to begin processing.
- The first hidden state $a^{\langle 1 \rangle}$ is computed based on this zero input.

- **Step 1: Predict the first word ($t = 1$)**

- The model applies a **Softmax function** to predict the first word $\hat{y}^{\langle 1 \rangle}$.
- Softmax assigns a probability to each word in the vocabulary.
- Example: $P(\text{"dogs"}) = 0.72$, $P(\text{"cats"}) = 0.15$, $P(\text{"have"}) = 0.08, \dots$

- **Step 2: Predict the second word ($t = 2$)**

- The correct first word $y^{\langle 1 \rangle}$ ("dogs") is now used as input $x^{\langle 2 \rangle}$.
- The model updates its hidden state $a^{\langle 2 \rangle}$.
- It then predicts $\hat{y}^{\langle 2 \rangle}$, assigning probabilities to words like "have," "run," or "eat."

- **Subsequent Steps**

- The process repeats, using the correct previous word as input.
- Each time step updates the hidden state $a^{\langle t \rangle}$.
- At the final step, the model predicts **End-of-Sentence (EOS)**: $\langle \text{EOS} \rangle$.

Training the RNN

- The model uses a **Softmax loss** at each time step t :

$$\mathcal{L}^{\langle t \rangle} = -\log P(y^{\langle t \rangle} | y^{\langle 1 \rangle}, \dots, y^{\langle t-1 \rangle})$$

- The **total Loss** is computed as the sum of losses across all time steps:

$$\mathcal{L} = \sum_{t=1}^{T_y} \mathcal{L}^{\langle t \rangle}$$

- **Training objective:** Minimize the total loss over all training examples.
- The model learns to predict the next word given all previous words.

Training the RNN

- **Example:** "Dogs have an average lifespan of 12 years." item The RNN sequentially predicts:

$$P(\text{Dogs})$$

$$P(\text{have}|\text{Dogs})$$

$$P(\text{an}|\text{Dogs, have})$$

$$P(\text{average}|\text{Dogs, have, an})$$

$$P(\text{lifespan}|\text{Dogs, have, an, average})$$

$$P(\text{of}|\text{Dogs, have, an, average, lifespan})$$

$$P(12|\text{Dogs, have, an, average, lifespan, of})$$

$$P(\text{years}|\text{Dogs, have, an, average, lifespan, of, 12})$$

$$P(\langle \text{EOS} \rangle |\text{Dogs, have, an, average, lifespan, of, 12, years})$$

Challenges in Training RNNs: Vanishing and Exploding Gradients

● RNNs and Long-Term Dependencies:

- RNNs are used for language modeling and sequence-based tasks.
- Some language patterns require remembering information from much earlier in the sequence.
- Example: In the sentence **"The cat, which already ate a bunch of food, was full."** The model must remember "cat" (singular) to correctly use "was." However, long gaps in dependencies make this difficult for basic RNNs.

● Vanishing gradient problem:

- In deep neural networks, gradients diminish as they propagate back.
- In RNNs, gradients must propagate through many time steps, making it hard to update earlier layers. This limits the model's ability to learn long-term dependencies.

● Exploding gradient problem:

- Sometimes, instead of vanishing, gradients **grow exponentially**.
- This causes weight updates to become too large, leading to unstable learning. A sign of exploding gradients: parameters become NaN.
- **Solution: gradient clipping** - If gradients exceed a threshold, they are rescaled to prevent excessive growth.

Solution to Vanishing Gradient Problem

- **Gated Architectures** - to address the vanishing gradient problem in RNN, researchers introduced **gated mechanisms** that control information flow.
- Two widely used architectures:
 - **Gated Recurrent Unit (GRU)**
 - **Long Short-Term Memory (LSTM)**
- GRUs and LSTMs enable RNNs to learn long-range dependencies effectively.
- These models help prevent information from fading as the sequence progresses.
- They are widely used in NLP tasks, speech recognition, and time-series prediction.

RNN vs. GRU

Example Sentence: "The cat, which already ate . . . , was full."

Recurrent Neural Network (RNN)

- Processes input sequentially, updating hidden state at each step.
- Uses a simple activation function (e.g., **tanh**) to update hidden states.
- **Problem:** Struggles with long-range dependencies due to vanishing gradients.
- In the example sentence, the RNN may forget "cat" by the time it predicts "was full."

Gated Recurrent Unit (GRU)

- Uses **gates** to control information flow and prevent vanishing gradients.
- **Reset gate** (r_t) decides how much past information to forget.
- **Update gate** (z_t) decides how much past information to retain.
- **Advantage:** Allows long-term dependencies to be maintained.
- In the example sentence, the GRU retains information about "cat," enabling it to predict "was full" correctly.

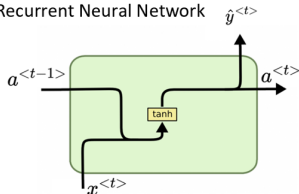
Gated Recurrent Unit (GRU)

● Gated Recurrent Unit (GRU)

- Uses **update gate** and **reset gate** to control how much past information is retained.
- The update gate decides how much past information to keep.
- The reset gate helps discard irrelevant information.

RNN

Recurrent Neural Network



$$a^{<t>} = g(W_a[a^{<t-1>}, x^{<t>}] + b_a)$$

GRU

Gated Recurrent Unit

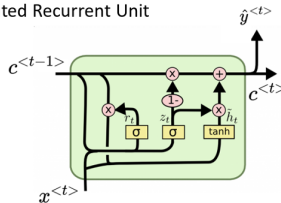


Figure: Comparison of RNN and GRU architectures

Gated Recurrent Unit (GRU)

- **Hidden State:**

- $c^{(t)}$ – **Current hidden state** (also called cell state).
- $c^{(t-1)}$ – **Previous hidden state**.

- **Input and Output:**

- $x^{(t)}$ – **Input** at time step t .
- $\hat{y}^{(t)}$ – **Predicted output** at time step t .

- **Gates:**

- r_t – **Reset gate**, decides how much of the past hidden state to forget.
- z_t – **Update gate**, determines how much of the previous hidden state is retained.

- **Intermediate Computations:**

- $\tilde{h}_t$ – **Candidate hidden state**, computed using the **reset gate**.
- $1 - z_t$ – **Complement of the update gate**, controlling how much new information to use.

- **Activation Functions:**

- σ – **Sigmoid activation function**, used in both the reset and update gates.
- $\tanh$ – **Tanh activation function**, used for computing the candidate hidden state.

Long Short-Term Memory (LSTM)

- **What is LSTM?**

- **Long Short-Term Memory (LSTM)** is an advanced RNN architecture designed to handle **long-term dependencies**.
- Unlike standard RNNs, LSTMs use a **cell state** to store information over long sequences.
- This helps solve the **vanishing gradient problem**, allowing LSTMs to retain important information for longer periods.

- **LSTM Structure: Three Gates for Memory Control**

- **Forget Gate (f_t):**

- Determines which part of the previous cell state should be discarded.

- **Equation:**

$$f_t = \sigma(W_f[a^{(t-1)}, x^{(t)}] + b_f)$$

- **Input Gate (i_t):**

- Controls how much new information should be added to the cell state.

- **Equation:**

$$i_t = \sigma(W_i[a^{(t-1)}, x^{(t)}] + b_i)$$

- **Output Gate (o_t):**

- Determines what to output at each time step.

- **Equation:**

$$o_t = \sigma(W_o[a^{(t-1)}, x^{(t)}] + b_o)$$

LSTM Example: Sentence Processing

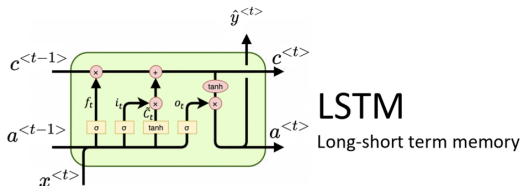
Example Sentence: "The cat, which already ate, was full."

- **How LSTM Processes This Sentence:**

- LSTM must retain the subject "**cat**" to correctly generate "**was**" instead of "**were**".
- The **forget gate** might discard unnecessary details like "**which already ate**".
- The **input gate** ensures the model remembers that "**cat**" is singular.
- The **output gate** then ensures the correct verb form "**was**" is produced.

- **Mathematical Representation:**

- **Cell State Update:** $c^{(t)} = f_t \cdot c^{(t-1)} + i_t \cdot \tilde{c}_t$
- **Hidden State Update:** $a^{(t)} = o_t \cdot \tanh(c^{(t)})$



Long Short-Term Memory (LSTM)

- **Cell State and Hidden State:**

- $c^{(t)}$ – **Cell state**, stores long-term memory.
- $c^{(t-1)}$ – **Previous cell state**, influences the current state.
- $a^{(t)}$ – **Hidden state** (short-term memory).
- $a^{(t-1)}$ – **Previous hidden state**, used in computations.

- **Input and Output:**

- $x^{(t)}$ – **Input** at time step t .
- $\hat{y}^{(t)}$ – **Predicted output** at time step t .

- **Gates:**

- f_t – **Forget gate**, decides how much of the previous cell state to retain.
- i_t – **Input gate**, controls how much new information is added to the cell state.
- o_t – **Output gate**, determines how much of the cell state should be output as the hidden state.

- **Intermediate Computations:**

- $\tilde{c}_t$ – **Candidate cell state**, computed using the **input gate**.
- $c^{(t)} = f_t \cdot c^{(t-1)} + i_t \cdot \tilde{c}_t$ – **Cell state update equation**.
- $a^{(t)} = o_t \cdot \tanh(c^{(t)})$ – **Hidden state update equation**.

- **Activation Functions:**

- σ – **Sigmoid**, used in the gates to regulate information flow.
- $\tanh$ – **Tanh**, used for candidate cell state and hidden state updates.

Bidirectional LSTM (BiLSTM)

● BiLSTM

- BiLSTM is an extension of LSTM that processes sequences in **both directions**.
- It consists of two LSTM layers:
 - **Forward LSTM**: Processes the sequence from left to right.
 - **Backward LSTM**: Processes the sequence from right to left.
- The outputs from both directions are concatenated before making the final prediction.

● Advantages of BiLSTM:

- Captures **both past and future context** for better sequence understanding.
- Useful in **Natural Language Processing (NLP), Speech Recognition, and Time-Series Analysis**.
- Performs better than standard LSTMs when **context from both directions is important**.

